Aperture — API Specification

Version: v1.0

Document Type: API Specification

Related Documents

- Technical Architecture
- Low-Level Design
- Database Design

---

1. Purpose

The API is the contract between Aperture's frontend, backend, and any future third-party consumers.

This document defines:

- API design philosophy
- Naming conventions
- Authentication
- Versioning
- Request and response standards
- Error handling
- Pagination
- Endpoint organization

Detailed request and response schemas will be generated automatically from FastAPI's OpenAPI specification.

This document explains **why** the API is designed the way it is.

---

2. API Design Principles

The API should be:

- Predictable
- Consistent
- Versioned
- Resource-oriented
- Easy to document

- Easy to evolve
- Backward compatible whenever practical

Consumers should not need to understand internal implementation details.

---

3. Architectural Style

Aperture adopts a RESTful architecture.

Reasons:

- Broad industry adoption
- Excellent tooling
- Strong FastAPI support
- Clear resource modeling
- Interview familiarity

Future capabilities such as GraphQL or gRPC may be introduced for specialized workloads, but REST remains the primary public interface.

---

4. Versioning Strategy

All endpoints are versioned.

Example:

```
/api/v1/
```

Major versions are introduced only when breaking changes cannot be avoided.

Minor feature additions should remain backward compatible.

---

5. Authentication

Protected endpoints require authentication.

Authentication uses:

- JWT access tokens
- Refresh tokens

- OAuth (Google initially)

Authentication should remain stateless.

Administrative APIs require elevated authorization.

---

6. API Organization

Endpoints are grouped by domain.

```
/api/v1

/auth
/users
/movies
/tv
/people
/search
/reviews
/ratings
/lists
/discovery
/streaming
/notifications
/admin
```

This mirrors the backend module structure.

---

7. Resource Naming

Resources use nouns.

Examples:

Good

```
GET /movies
GET /reviews
POST /lists
```

Avoid verbs in resource names whenever possible.

Nested resources should represent ownership rather than convenience.

Example:

```
GET /movies/{id}/reviews
```

---

8. HTTP Methods

GET

Retrieve resources.

POST

Create resources.

PUT

Replace an existing resource.

PATCH

Partial update.

Preferred for most user edits.

DELETE

Delete or soft-delete a resource.

---

9. Standard Response Structure

Successful responses should follow a consistent envelope.

Example:

```
{
  "data": {},
  "meta": {},
  "links": {}
}
```

Where appropriate:

- `data` contains the primary resource
- `meta` includes pagination or counts
- `links` supports navigation

Simple endpoints may return only the resource itself when additional metadata is unnecessary.

---

10. Error Handling

Errors should be predictable and machine-readable.

Representative structure:

```
{
  "error": {
    "code": "RESOURCE_NOT_FOUND",
    "message": "Movie not found.",
    "details": {}
  }
}
```

Internal implementation details should never be exposed.

---

11. Status Codes

Common responses include:

- 200 OK
- 201 Created
- 204 No Content
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found

- 409 Conflict
- 422 Validation Error
- 429 Too Many Requests
- 500 Internal Server Error

The same situation should always produce the same status code.

---

12. Pagination

Collection endpoints should support pagination.

Default strategy:

Cursor pagination:

- Reviews
- Notifications
- Activity feeds
- Discovery feeds

Offset pagination:

- Administrative endpoints
- Search results
- Static reference data

Pagination metadata should be included in responses where appropriate.

---

13. Filtering

Filtering should be composable.

Examples:

Movies

- genre
- year
- runtime
- language
- streaming provider

Reviews

- rating
- spoiler
- sort

Lists

- owner
- visibility

New filters should not require redesigning existing endpoints.

---

14. Sorting

Sorting should be explicit.

Examples:

- popularity
- release date
- rating
- creation date
- relevance
- alphabetical

Descending order should be the default where appropriate.

---

15. Search

Search remains a dedicated resource.

Example:

```
GET /search
```

Supported categories:

- Movies
- TV Shows
- People
- Users

- Lists

Future semantic search should not require endpoint changes.

---

16. Rate Limiting

Public endpoints:

Moderate rate limits.

Authentication endpoints:

Strict limits.

Administrative endpoints:

Role-based limits.

Limits should be enforced consistently and communicated through standard response headers where possible.

---

17. Idempotency

Idempotency should be preserved where practical.

Examples:

- Repeating a DELETE should not corrupt state.
- Refresh token requests should rotate tokens safely.
- Duplicate follow requests should not create multiple relationships.

---

18. Documentation

FastAPI automatically generates:

- OpenAPI specification
- Swagger UI
- ReDoc

These generated artifacts become the authoritative implementation reference.

This document remains the architectural guide.

---

19. Endpoint Inventory

The API surface is organized by module.

Authentication

- Register
- Login
- Logout
- Refresh
- OAuth

Users

- Profile
- Preferences
- Followers
- Following

Metadata

- Movies
- TV Shows
- People
- Collections
- Genres

Reviews

- Reviews
- Ratings
- Likes

Lists

- Lists

- Watchlists
- List Items

Search

- Search
- Autocomplete

Discovery

- Homepage
- Trending
- Recommendations
- Editorial Collections

Streaming

- Availability
- Providers

Notifications

- Notifications
- Preferences

Administration

- Moderation
- Feature Flags
- Jobs
- Audit Logs

Detailed request and response models are intentionally omitted from this document because they are maintained automatically through OpenAPI generation.

---

20. Future Evolution

Potential additions include:

- GraphQL gateway
- Public developer API
- API keys
- Webhooks
- Batch endpoints
- Real-time subscriptions

The current API should remain stable as these capabilities are introduced.

---

21. Interview Discussion Points

Be prepared to explain:

- Why REST instead of GraphQL?
- Why generate OpenAPI documentation automatically?
- Why use cursor pagination for feeds?
- How do you version APIs without breaking clients?
- Why organize endpoints by domain?
- How would you expose a public API while protecting internal endpoints?

---

22. Summary

The Aperture API is designed around consistency, domain boundaries, and long-term maintainability.

Rather than tightly coupling clients to backend implementation details, the API exposes a stable resource-oriented interface that can evolve alongside the platform while remaining intuitive for consumers and straightforward to document through FastAPI's OpenAPI generation.